

Department of Electrical & Computer Engineering
First Semester 2025/2026
HW Design Lab
Project#1 – Design Verification (APB FIFO Verification using UVM)

Instructors: Abdellatif Abu-Issa and Ayman Hroub
Teaching Assistants: Raha Zabadi, Jehad Halahala, and Tala Abahra

1. Design Overview

Your task is to verify a buggy RTL design of an APB-based FIFO using the UVM methodology. You will build a complete UVM testbench, including an APB agent, register model, scoreboard, and coverage collection.

2. Design Overview

The DUT is a parameterized synchronous FIFO accessed through an APB3 slave interface. Software can push bytes into the FIFO via the DATA register and pop them by reading the same register. The design includes programmable thresholds, control bits, and sticky status flags.

Main Parameters

- WIDTH = 8 bits
- DEPTH = 16 entries

APB Interface

PSEL, PENABLE, PWRITE, PADDR[7:0], PWDATA[31:0], PRDATA[31:0], PREADY, PSLVERR

Register Map

Offset	Name	Access	Description
0x00	CTRL	RW	Bit[0] EN, Bit[1] CLR, Bit[2] DROP_ON_FULL
0x04	THRESH	RW	Bit[7:0] ALMOST_FULL_TH, Bit[15:8] ALMOST_EMPTY_TH
0x08	STATUS	RO/RC	Empty/Full/Almost/Overflow/Underflow/Count
0x0C	DATA	RW	Push or pop one byte

Functional Rules

1. FIFO operations depend on CTRL.EN.
2. Writing CTRL.CLR clears FIFO and sticky flags.
3. DATA writes push data when not full; reads pop data when not empty.
4. Overflow/Underflow flags are sticky until cleared.
5. DROP_ON_FULL defines whether to drop or error on full.
6. ALMOST_FULL and ALMOST_EMPTY depend on programmable thresholds.

3. What You Must Build (UVM Testbench)

- APB master agent (driver, sequencer, monitor)
- Register model (UVM_REG)
- Scoreboard with reference FIFO model
- Functional coverage
- Directed and random sequences covering all operations and errors

Deliverables

- UVM testbench source code
- Coverage and report summary
- Short bug report (observed failures and reasoning)

4. Buggy RTL (apb_sync_fifo.sv)

The following RTL contains several functional bugs intentionally added for verification purposes. You must identify them through simulation and coverage-driven verification. Do NOT modify the RTL.

```
//------------------------------------------------------------------------
---------
// APB3-accessible synchronous FIFO (WIDTH=8, DEPTH=16) — intentionally
buggy
//------------------------------------------------------------------------
---------
module apb_sync_fifo #(
  parameter int WIDTH = 8,
  parameter int DEPTH = 16,
  localparam int AW    = $clog2(DEPTH)
)(
  input  logic        PCLK,
  input  logic        PRESETn,
  input  logic        PSEL,
  input  logic        PENABLE,
  input  logic        PWRITE,
  input  logic [7:0]  PADDR,
  input  logic [31:0] PWDATA,
  output logic [31:0] PRDATA,
  output logic        PREADY,
  output logic        PSLVERR
);
```

```
// ---- APB constants ----
localparam CTRL_O   = 8'h00;
localparam THRESH_O = 8'h04;
localparam STATUS_O = 8'h08;
localparam DATA_O   = 8'h0C;

// ---- registers ----
logic        en, clr, drop_on_full;
logic [7:0]  almost_full_th, almost_empty_th;
logic        empty, full, almost_full, almost_empty;
logic        overflow, underflow; // sticky
logic [AW:0] count;               // allows DEPTH value

// FIFO storage
logic [WIDTH-1:0] mem [0:DEPTH-1];
logic [AW-1:0] wptr, rptr;

// APB default
assign PREADY  = 1'b1;     // zero-wait state
assign PSLVERR = 1'b0;

// Async reset
always_ff @(negedge PRESETn or posedge PCLK) begin
  if (!PRESETn) begin
    en <= 1'b0;
    clr <= 1'b0;
    drop_on_full <= 1'b0;
    almost_full_th <= DEPTH-1;
    almost_empty_th <= 8'd1;

  end else begin
    if (PSEL && !PENABLE && PWRITE) begin // setup phase latching
      unique case (PADDR)
        CTRL_O: begin
          en           <= PWDATA[0];
          clr          <= PWDATA[1];   // self-clear handled below
          drop_on_full <= PWDATA[2];
        end
        THRESH_O: begin
          almost_full_th  <= PWDATA[7:0];
          almost_empty_th <= PWDATA[15:8];
        end
        default: ;
      endcase
    end

    // self-clear of CLR
    if (clr) clr <= 1'b0;


    if (PSEL && PENABLE && !PWRITE) begin
      overflow <= 1'b0;
      underflow <= 1'b0;
    end
  end
end
```

```
  // STATUS read mux
  always_comb begin
    PRDATA = 32'd0;
    unique case (PADDR)
      STATUS_O: PRDATA = {18'd0, count[7:0], underflow, overflow,
almost_empty, almost_full, full, empty};
      DATA_O:   PRDATA = {24'd0, mem[rptr]};
      CTRL_O:   PRDATA = {29'd0, drop_on_full, clr, en};
      THRESH_O: PRDATA = {16'd0, almost_empty_th, almost_full_th};
      default:  PRDATA = 32'd0;
    endcase
  end

  // FIFO control
  logic push, pop;
  assign push = (PSEL && PENABLE && PWRITE && (PADDR==DATA_O));
  assign pop  = (PSEL && PENABLE && !PWRITE && (PADDR==DATA_O));

  // Count/flags
  always_ff @(posedge PCLK or posedge PRESETn) begin
    if (PRESETn) begin
      count <= '0;
      wptr  <= '0;
      rptr  <= '0;
      empty <= 1'b1;
      full  <= 1'b0;
    end else begin
      if (clr) begin
        count <= '0;
        wptr  <= '0;
        rptr  <= '0;
        empty <= 1'b1;
        full  <= 1'b0;
      end else if (en) begin
        // Push
        if (push) begin
          if (!full) begin
            mem[wptr] <= PWDATA[7:0];
            wptr <= wptr + 1'b1;
            count <= count + 1'b1;
          end else begin

            if (drop_on_full) begin
              // drop silently but set flag
              overflow <= 1'b1;
            end else begin
                  overflow <= 1'b1;
            end
          end
        end

        // Pop
        if (pop) begin
          if (!empty) begin
            rptr <= rptr + 1'b1;
            count <= count - 1'b1;
          end else begin
```

```
            underflow <= 1'b1;
          end
        end



        // Update empty/full
        empty <= (count == 0);
        full  <= (count == DEPTH-1);
      end
    end
  end


  always_ff @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
      almost_full  <= 1'b0;
      almost_empty <= 1'b1;
    end else begin
      almost_full  <= (count > almost_full_th);
      almost_empty <= (count <= almost_empty_th);
    end
  end

endmodule
```

---

5. Grading Policy

- UVM testbench architecture – 30 pts
- Stimulus quality – 25 pts
- Scoreboard and checks – 20 pts
- Coverage– 10 pts
- Short report including Bugs report – 15 pts